

JAVA INTERVIEW PREPARATION

CHAPTER 38

REST API DESIGN,
VALIDATION, AND SERIALIZATION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

REST API Design, Validation, and Serialization: Stable Contracts for Distributed Clients

Senior / Lead Java Developer Reading Chapter

A production REST API is a distributed contract, not a controller method exposed as JSON. Clients deploy independently, networks retry, caches reuse responses, schemas evolve, and failures must remain machine-readable. Senior design connects HTTP semantics, domain commands, validation, idempotency, pagination, concurrency control, and observability.

The important shift in thinking is that the API does not merely expose application code. It defines what remote consumers are allowed to assume. Once a mobile application, partner integration, or another service depends on a field or status code, changing that behavior becomes a distributed migration. A method can be refactored in one deployment; a public contract may need to survive several years of independently deployed clients.

This is why a technically valid controller can still be a poor API. It may serialize an entity successfully while leaking internal fields, return 200 for every outcome, or perform a non-idempotent action that becomes dangerous when a gateway retries it. Senior API design begins with failure, concurrency, evolution, and ownership rather than with annotations.

Model Resources, Not Database Tables

Resource paths should express the client-visible domain.

```
GET    /orders/{id}
POST   /orders
PATCH /orders/{id}
POST   /orders/{id}/cancellations
```

The cancellation subresource expresses a business action with its own authorization, idempotency, and audit semantics. Forcing every operation into CRUD can hide the domain; using arbitrary verbs in URLs can ignore useful HTTP behavior. Choose the clearest stable contract.

HTTP Method Semantics

GET and HEAD are safe: clients do not request state change. PUT is normally idempotent replacement at a known URI. DELETE is idempotent in intended effect. POST is not inherently idempotent. PATCH applies a partial modification with semantics defined by its media type and API.

Idempotent does not mean identical response on every call. Repeating DELETE may return 204 then 404 while the final resource state remains deleted.

Do not perform state-changing work in GET. Browsers, crawlers, prefetchers, and caches can issue it automatically.

Method semantics also influence what infrastructure is allowed to do. A proxy can cache a GET, a client can retry an idempotent PUT after a broken connection, and an observability platform can group requests by method and route. If the API labels every operation POST, the application must rebuild those semantics itself.

Consider an order replacement. A PUT to `/orders/42` says that the client knows the target identity and is supplying the desired representation. Repeating the request has the same intended result. A POST to `/orders` usually asks the server to allocate a new identity, so repeating it may create a second order. POST

can still be made idempotent, but only by adding an explicit application mechanism such as an idempotency key.

Status Codes and Headers

Use status codes to convey protocol outcome:

200 OK	successful response with representation
201 Created	new resource, usually with Location
202 Accepted	accepted for asynchronous processing
204 No Content	success without response body
400 Bad Request	malformed or structurally invalid request
401 Unauthorized	authentication required or invalid
403 Forbidden	authenticated but not permitted
404 Not Found	resource unavailable or intentionally concealed
409 Conflict	current state conflicts with operation
412 Precondition	Failed conditional request failed
422 Unprocessable Content	well-formed but semantically invalid where adopted
429 Too Many Requests	rate limit
503 Service Unavailable	temporary incapacity

Contract meaning matters more than memorizing one universal mapping. Document error codes and retryability. Include [Location](#), [ETag](#), [Retry-After](#), caching, and correlation headers where appropriate.

Status codes operate at the protocol level while the response body explains the domain-level result. A 409 tells generic infrastructure that the request conflicts with current state; a stable problem code such as [ORDER_ALREADY_SHIPPED](#) tells the client which business conflict occurred. Returning 200 with `{ "success": false }` hides failure from gateways, metrics, retry libraries, and standard clients.

The distinction between 400 and 422 should be an organizational contract rather than a matter of personal taste. Some APIs use 400 for malformed JSON and 422 for a structurally readable request rejected by validation. Others consistently use 400 for both and distinguish them through problem codes. Either can work if documented and applied uniformly. By contrast, using 500 for expected validation or conflict is misleading because it describes a server defect and may trigger retries.

Headers are part of the representation contract. [Location](#) identifies a created resource; [WWW-Authenticate](#) describes an authentication challenge; [Retry-After](#) makes temporary throttling actionable; [Vary](#) prevents a cache from reusing a response selected for a different request header. They should be covered by contract tests, not treated as incidental framework output.

API DTOs

Do not expose JPA entities directly. Entities contain persistence identity, lazy associations, bidirectional graphs, internal fields, and change for database reasons.

```
public record OrderResponse(  
    UUID id,  
    String status,  
    BigDecimal total,  
    String currency,  
    Instant createdAt  
) {}
```

Explicit DTOs stabilize naming, control sensitive data, and allow mapping from several aggregates. Records are convenient when immutable data-carrier semantics fit.

Request and response DTOs should normally be distinct. A response may contain server-owned identifiers, audit timestamps, calculated totals, and links that a caller must never be allowed to submit. A create request contains only client-controlled input. Reusing one object for both directions makes over-posting easier and obscures ownership.

Mapping is not wasted boilerplate when it protects a contract boundary. It is the place to convert internal enum names, normalize money and timestamps, hide soft-delete flags, and combine data from multiple domain objects. Automated mappers can reduce repetition, but mappings that affect security or compatibility still require explicit tests.

Input Validation

```
public record CreateOrderRequest(  
    @NotBlank String customerId,  
    @NotEmpty List<@Valid LineRequest> lines,  
    @NotNull String idempotencyKey  
) {}
```

Bean Validation protects structural conditions. Domain validation requires current state and belongs in the application/domain layer. Database constraints close race windows.

```
request shape -> domain policy -> persistence constraint
```

Return field paths and stable error codes without echoing secrets. Do not reveal whether a protected account exists when authorization policy requires concealment.

The three validation layers solve different problems. Bean Validation rejects impossible request shapes before application work begins. Domain policy answers questions such as whether an order in **SHIPPED** state can be cancelled. A unique database constraint protects the final race in which two valid-looking requests concurrently claim the same business key. Removing any layer creates either poor feedback or a correctness gap.

Validation must also have cost limits. A request containing one million line items can satisfy type rules while exhausting memory or CPU. Apply transport limits before deserialization, collection-size constraints during validation, and business limits in the use case. A validator should not quietly issue hundreds of database queries; bulk-load required state or move expensive checks into an application service where transaction and timeout behavior are visible.

Deserialization Is an Attack Boundary

Limit request size, nesting depth where supported, collection cardinality, number length, and accepted media types. Reject unknown fields when strict contracts and client safety justify it, or ignore them deliberately for forward compatibility. The choice must be documented.

Avoid enabling unsafe polymorphic deserialization for untrusted payloads. Whitelist concrete API types through explicit contracts. Configure date/time and number handling centrally and test it.

Null, Missing, and Empty

For updates, missing and explicit null can have different meaning:

```
field absent -> leave unchanged  
field null   -> clear value  
field value  -> replace value
```

A Java nullable field cannot always distinguish these states. Use dedicated patch models, JsonNullable-like wrappers where appropriate, JSON Merge Patch, or JSON Patch with a clear media type.

Collections should usually be returned as empty arrays rather than null. Consistency reduces client branching.

Idempotency

For retryable POST operations, accept an idempotency key scoped to the authenticated client and operation.

```
client + operation + key -> request fingerprint + stored outcome
```

The server atomically records or claims the key with the business operation. A duplicate with the same fingerprint returns the original outcome; the same key with a different payload is rejected.

An in-memory map is insufficient across nodes and restarts. Define retention, concurrent duplicates, in-progress behavior, failure recovery, and whether 5xx results are replayed.

The atomicity requirement is the hard part. If the order commits but the idempotency record does not, a retry can create a duplicate. If the record says complete before the order commits, a retry can receive a result for work that never became durable. Common designs store the idempotency claim and business change in one database transaction, or use a durable state machine with reconciliation when the stores cannot share a transaction.

```
ABSENT --atomic claim--> IN_PROGRESS --business commit--> SUCCEEDED
                        |                               |
duplicate waits/retries ---+                         +--> replay outcome
                        |
                        +--recoverable failure--> FAILED or claim expiry
```

Store a canonical request fingerprint so accidental key reuse with a different payload becomes a conflict, not a replay of unrelated data. Decide whether response bodies are retained, reconstructed from a resource identifier, or redacted after a shorter period than the key itself. Idempotency is a consistency feature with storage, privacy, and lifecycle consequences.

Optimistic Concurrency with ETags

Return an ETag representing a resource version. A client sends *If-Match* on update.

```
GET /orders/7 -> ETag: "v12"
PATCH /orders/7 If-Match: "v12"
current is v13 -> 412 Precondition Failed
```

This prevents lost updates without holding a database transaction across user think time. The ETag must represent the relevant state and should not expose sensitive internals.

An ETag used for concurrency control should be a strong validator for the state being changed. It can be derived from a JPA version field but should be formatted as an opaque HTTP value. The controller translates *If-Match* into a precondition and the persistence layer still uses an atomic version check. Checking the version in Java and later issuing an unconditional update leaves a race window.

```
@PatchMapping("/orders/{id}")
ResponseBody<OrderResponse> update(
    @PathVariable UUID id,
    @RequestHeader("If-Match") String expected,
    @Valid @RequestBody UpdateOrderRequest request) {
    Version version = etags.parse(expected);
    OrderView updated = service.update(id, version, request);
    return ResponseEntity.ok()
        .eTag(etags.format(updated.version()))
        .body(mapper.toResponse(updated));
}
```

Conditional GET uses *If-None-Match* and normally produces 304 when unchanged. Conditional mutation uses *If-Match* and normally produces 412 when the precondition fails. The two mechanisms share validators

but solve bandwidth reuse and lost-update prevention respectively.

Pagination

Offset pagination is simple but large offsets can be expensive and concurrent inserts can duplicate or skip results. Cursor or keyset pagination uses a stable ordered position.

```
ORDER BY created_at, id
next cursor encodes last created_at and id
```

Always include a deterministic tie-breaker. Treat cursor contents as an opaque signed or validated token. Define maximum page size and filtering/index requirements.

Avoid returning total count automatically when it requires a costly full query. Make it optional or provide an estimate if product semantics allow.

A cursor is not merely a base64-encoded page number. It represents a position in a declared ordering and often includes normalized filter or sort context. Sign or validate it so callers cannot inject unsupported query state. If the ordering column is mutable, rows can still move across the boundary; immutable creation time plus a unique ID is usually easier to reason about than a frequently changing score.

Keyset pagination trades random page access for predictable database work. That is often appropriate for feeds and operational lists, while offset pagination remains reasonable for small administrative datasets. The senior answer is not "cursor is always better" but that the choice follows access pattern, index design, consistency expectations, and user experience.

Versioning and Compatibility

Prefer additive evolution: add optional response fields, accept older request shapes when safe, and avoid changing existing field meaning. Clients should tolerate documented additions.

Breaking changes may use URI, header, or media-type versioning. The organization needs one consistent strategy, deprecation telemetry, communication, and removal policy.

Database schema version and API version are different. A service can translate one external contract across several internal migrations.

Compatibility must be considered in both directions. Adding an optional response field is usually safe only if clients tolerate unknown fields. Adding a required request field is breaking for old clients. Renaming an enum value or changing a number from cents to dollars is breaking even if the JSON type remains unchanged.

A robust removal process starts with usage telemetry. Mark a field or version deprecated, communicate a deadline, identify remaining clients, and remove only after the support policy permits it. Versioning every minor addition creates unnecessary parallel APIs; refusing to version genuine semantic breaks silently transfers migration risk to clients.

Consumer-driven contract tests can reveal that a provider change violates a known consumer expectation, but they are not a substitute for API ownership. The provider must still control supported contracts, security rules, and retirement policy rather than accepting every historical consumer behavior forever.

Caching

Use [Cache-Control](#), ETag, and Last-Modified according to resource semantics. Personalized or authorization-sensitive responses require correct private/no-store policy and cache-key variation.

Do not cache error responses or authenticated content accidentally at a shared proxy. Ensure [Vary](#) reflects representation selection where needed.

Caching has two independent questions: may a response be stored, and may a stored representation be reused for this request? `Cache-Control` answers storage and freshness; validators such as ETag support revalidation. A personalized response marked `private` can be stored by a user agent but not a shared cache. `no-store` prohibits storage and is stronger than merely requiring revalidation.

Spring MVC can attach cache policy and ETags through `ResponseEntity`. A shallow content hash can save network bandwidth, but because the response must first be generated to calculate it, it does not eliminate controller or serialization work. A domain version known before loading an expensive representation can support a more efficient conditional check.

Asynchronous Operations

For long work, return 202 with a status resource:

```
POST /exports -> 202 Location: /operations/abc
GET /operations/abc -> running | completed | failed
```

Define cancellation, expiry, result download authorization, retry, and duplicate submission. A request thread should not wait minutes for a batch that survives deployment.

The operation resource is a state machine, not just a polling endpoint. It should expose durable status, terminal failure information safe for the caller, progress only if it is meaningful, and a result link once available. Returning 202 without a way to discover completion has merely hidden the timeout.

Submission and worker execution should be joined by durable messaging or a database-backed queue. Publishing only after an uncoordinated database commit can lose work; publishing before commit can expose work whose inputs are not visible. Transactional outbox or equivalent coordination is a common answer when the database and broker do not share a transaction.

Spring MVC Contract Implementation

Keep the controller thin, but not meaningless. It owns HTTP translation: media types, request binding, status, headers, and conversion between API DTOs and an application command. The application service owns the use case and transaction. The domain owns invariant decisions.

```
@PostMapping
ResponseEntity<OrderResponse> create(
    @RequestHeader("Idempotency-Key") String key,
    @Valid @RequestBody CreateOrderRequest body,
    Authentication caller) {
    CreateOrder command = mapper.toCommand(body, caller.getName(), key);
    CreatedOrder result = application.create(command);
    return ResponseEntity.created(URI.create("/orders/" + result.id()))
        .eTag(etags.format(result.version()))
        .body(mapper.toResponse(result));
}
```

This separation also makes testing precise. A web-slice test verifies binding, validation, serialization, status, and headers. An application test verifies idempotency and transactional behavior. An end-to-end contract test verifies the assembled security and error contract. Mocking the service in every test cannot prove that duplicate requests are safe.

Common Senior-Level Traps

The first trap is equating REST with URL naming. A beautiful resource path does not compensate for unsafe retry behavior, unstable schemas, or incorrect authorization. The second is exposing persistence models and assuming Jackson annotations will permanently protect the contract. The third is treating validation as a controller concern while ignoring concurrent database enforcement.

Other frequent traps include unbounded page sizes, offset pagination on a rapidly changing large table, totals calculated for every request, and a global cache that ignores tenant or authorization context. A technically successful response can still be a production incident if it leaks another tenant's representation or creates duplicates after a timeout.

A Strong Interview Answer

I treat a REST API as a long-lived distributed contract. I begin with resource and command semantics, choose HTTP methods and status codes that infrastructure understands, and keep request and response DTOs independent from entities. Validation occurs at transport, domain, and database layers. For retried creation I use a durable idempotency key tied to caller, operation, and request fingerprint; for concurrent updates I use ETag and an atomic version check. At scale I use deterministic keyset pagination, explicit cache policy, and operation resources for long work. I then contract-test serialization, headers, security, failure bodies, and compatibility, and observe route templates and stable error codes without leaking sensitive payloads.

Production Perspective

Record route templates rather than raw IDs in metric labels. Measure status family, latency, payload size, validation failures, idempotency replay, concurrency conflicts, and client version. Avoid logging entire payloads by default.

Contract tests and generated specifications help, but verify actual serialization, security, headers, and error bodies. A schema that says `required` while production returns null is not a contract.

Interview-Ready Summary

REST design uses HTTP semantics to create a stable distributed contract. Separate API DTOs from persistence entities, validate at structural, domain, and database layers, and design retries through idempotency rather than hope.

Use conditional requests for lost-update prevention, deterministic cursor pagination at scale, explicit error and caching semantics, and additive evolution where possible. Long operations should become resources of their own. Production observability must control cardinality and sensitive data.

Revision Notes

- Model client-visible resources and business operations deliberately.
- GET must not perform requested state changes.
- Idempotency concerns final effect, not identical responses.
- Use protocol status codes and headers consistently.
- API DTOs protect contracts from persistence changes.
- Bean Validation protects shape; domain rules need application state.
- Database constraints close concurrent validation races.
- Deserialization needs size, type, and complexity limits.
- Decide and document unknown-field behavior.
- Missing, null, and empty can have distinct update meanings.
- Distributed POST retries need durable idempotency records.
- Bind idempotency keys to client, operation, and request fingerprint.
- ETag plus If-Match prevents lost updates.
- Cursor pagination needs stable ordering and a tie-breaker.
- Large total counts may be expensive.
- Prefer additive compatible API evolution.
- API and database versions are separate concerns.
- Secure caching of personalized responses explicitly.
- Represent long-running work with 202 and operation resources.
- Observe route templates, outcomes, sizes, and contract failures safely.